

واحد محاسبات

- محدودیت زمان: ۳۰۰ میلی‌ثانیه
- محدودیت حافظه: ۱۰ مگابایت

در این مسئله قرار است یک واحد محاسبات ساده پیاده‌سازی کنید. این واحد محاسبه، تعدادی ثبات (Register) دارد که می‌توان یک مقدار اعشاری را به‌طور مستقیم روی آن‌ها قرار داد، مقدار آن‌ها را چاپ کرد و با آن‌ها محاسبات انجام داد.



دقت کنید که شماره‌ی ثبات‌ها از صفر شروع می‌شود و به‌صورت یک عدد طبیعی به آن اشاره می‌شود. همچنین اجزای هر دستور با یک فاصله از هم جدا شده‌اند.

به‌طور ساده، ثبات‌ها را خانه‌هایی از حافظه در نظر بگیرید که در دسترس واحد محاسبه هستند. هر خانه یک عدد ممیز شناور ذخیره می‌کند و با شماره‌اش به آن اشاره می‌کنیم.

دستورات و توضیحات آن‌ها

برای هر دستور محاسباتی، فرمت کلی، یک نمونه و عملیات معادل آن در سه خط آورده شده است.

قرار دادن یک عدد روی یک ثبات

این دستور مقدار داده شده را روی ثبات خواسته شده قرار می دهد.

```
1 | Format:    load <reg> <number>
2 | Example:  load 0 0.23
3 | Operation: reg[0] = 0.23
```

پایان

این دستور به معنای پایان کار است و برنامه باید خاتمه بیابد.

```
1 | end
```

چاپ مقدار فعلی یک ثبات

مقدار ثبات خواسته شده باید چاپ شود. خروجی ها تا **چهار رقم** اعشار به روش گرد کردن گزارش شوند.

```
1 | print <reg>
2 | print 3
```

عملیات اصلی

در این دستورات، دو ثبات اول، مبدأ هستند و ثبات آخر مقصد است.

جمع

```
1 | Format:    add <reg1> <reg2> <reg3>
2 | Example:  add 0 1 2
3 | Operation: reg[2] = reg[0] + reg[1]
```

تفریق

```
1 | Format:    sub <reg1> <reg2> <reg3>
2 | Example:  sub 0 1 2
3 | Operation: reg[2] = reg[0] - reg[1]
```

ضرب مقادیر

```
1 | Format:    mul <reg1> <reg2> <reg3>
2 | Example:  mul 0 1 2
3 | Operation: reg[2] = reg[0] * reg[1]
```

تقسیم

در تقسیم اگر مخرج صفر بود، نباید دستور اجرا شود. بدون خطا از این دستور عبور کنید.

```
1 | Format:    div <reg1> <reg2> <reg3>
2 | Example:  div 0 1 2
3 | Operation: reg[2] = reg[0] / reg[1]
```

استفاده از بسط تیلور!

در این دستورات، باید حاصل تقریبی تابع خواسته شده با ورودی ثابت اول را در ثابت دوم بریزید. یک عدد حسابی (n) هم به شما داده می شود که مرتبه‌ی مورد نیاز برای محاسبه‌ی تقریبی به کمک بسط تیلور است.

فرمول تقریب با بسط تیلور برای توابع مورد نیاز در بخش بعدی آمده است.

سینوس

```
1 | Format:    sin <reg1> <reg2> <n>
2 | Example:  sin 0 1 10
3 | Operation: reg[1] = sin(reg[0]) // with order 10
```

کسینوس

```
1 | Format:    cos <reg1> <reg2> <n>
2 | Example:  cos 3 4 5
3 |
```

```
Operation: reg[4] = cos(reg[3]) // with order 5
```

نمایی

```
1 | Format:    exp <reg1> <reg2> <n>
2 | Example:  exp 1 2 3
3 | Operation: reg[2] = exp(reg[1]) // with order 3
```

بررسی خطاها

اگر دستوری به ثبات غیرقابل قبولی اشاره می‌کرد، نباید هیچ محاسبه‌ای برای آن دستور انجام شود و در خروجی این خط می‌آید:

INVALID REGISTER NUMBER

تقریب با بسط تیلور (حول نقطه‌ی صفر)

$$\sin x = \sum_{i=0}^{n-1} (-1)^i \frac{x^{2i+1}}{(2i+1)!}$$

$$\cos x = \sum_{i=0}^{n-1} (-1)^i \frac{x^{2i}}{(2i)!}$$

$$e^x = \sum_{i=0}^{n-1} \frac{x^i}{i!}$$

ورودی

در اولین خط ورودی، عدد حسابی n می‌آید که تعداد ثبات‌های ماشین است. در ابتدا همه‌ی ثبات‌ها مقدار صفر دارند.

در ادامه، در هر خط یک دستور می‌آید. تضمین می‌شود ساختار دستورات واردشده قابل قبول باشند و آخرین دستور، دستور پایان (end) باشد.

خروجی

هر دستوری که منجر به چاپ خروجی می‌شود، در یک خط خروجی‌اش چاپ می‌شود.

دقت شود که خروجی این دستورات باید به همان ترتیب دستورشان در ورودی آمده باشند.

مثال

ورودی نمونه

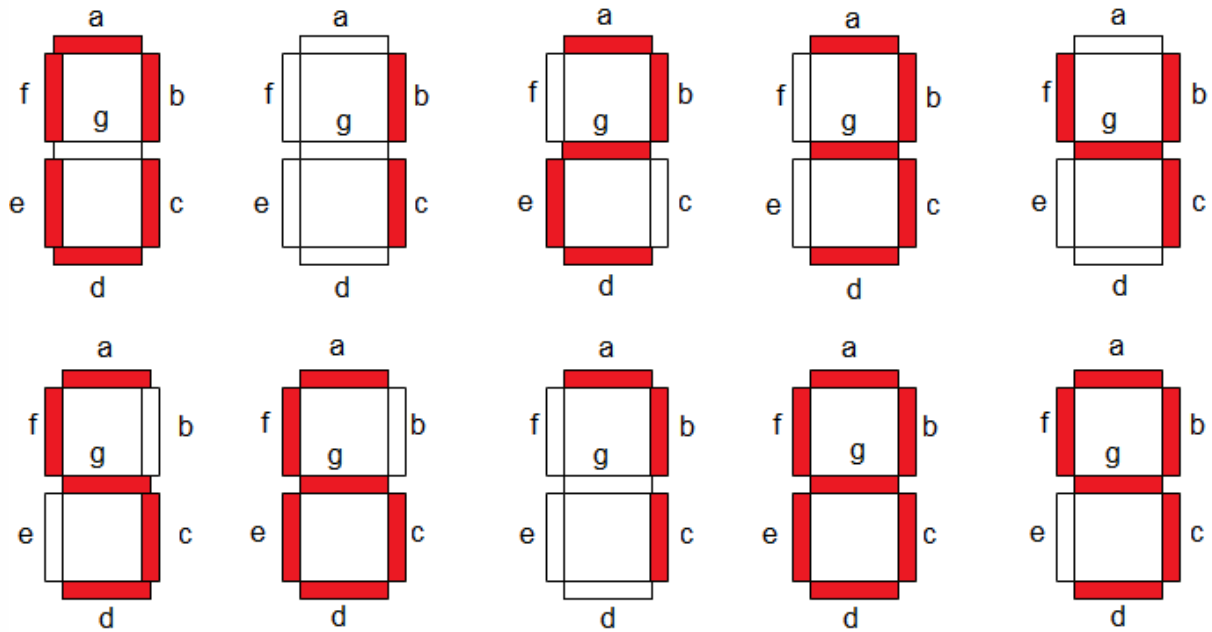
```
3
load 0 1.0
load 1 2.0
add 0 1 2
print 2
end
```

خروجی نمونه

```
3.0000
```

سون سگمنت

- محدودیت زمان: ۳۰۰ میلی‌ثانیه
- محدودیت حافظه: ۱۰ مگابایت



در این سؤال باید برنامه‌ای به زبان C بنویسید که یک رشته از ارقام (۰ تا ۹) را دریافت کرده و هر رقم را در قالب یک نمایشگر سگمنت LED در ابعاد ۹×۷ **کاراکتری** نمایش دهد.

ساختار نمایش هر رقم

هر رقم از ۷ سگمنت تشکیل شده‌است (مثل نمایشگرهای دیجیتال):

```
*****
*      *
*      *
*      *
*****
*      *
*      *
*      *
*****
```

Row 0: Top segment (T)

Rows 1- 4: Left and right upper segments (LU, RU)

Row 4: Middle segment (M)

Rows 5-8: Left and right lower segments (LL, RL)

Row 9: Bottom segment: Bottom segment (B)

الگوی روشن شدن سگمنت‌ها برای هر رقم

رقم	سگمنت‌های فعال
۰	T, B, LU, LL, RU, RL
۱	RU, RL
۲	T, RU, M, LL, B
۳	T, RU, M, RL, B
۴	LU, RU, M, RL
۵	T, LU, M, RL, B
۶	T, LU, M, LL, RL, B
۷	T, RU, RL
۸	T, M, B, LU, LL, RU, RL
۹	T, LU, RU, M, RL, B

ورودی

ورودی شامل یک رشته از ارقام ۰ تا ۹ است که در یک خط و بدون فاصله می‌آید.

- طول رشته می‌تواند هر مقدار مثبتی باشد.
- رشته شامل کاراکترهای غیررقمی نیست.
- طول رشته حداکثر ۱۰۰۰ کاراکتر است.

خروجی

خروجی برنامه باید نمایش گرافیکی ارقام را به این صورت نشان دهد:

- هر رقم در یک شبکه‌ی ۹×۷ نمایش داده شود.
- ارقام به صورت افقی در کنار هم چاپ شوند.
- بین هر دو رقم ۳ فاصله (سه کاراکتر space) وجود داشته باشد.

- هیچ خط اضافه‌ای قبل یا بعد از خروجی چاپ نشود.

خروجی شامل ۹ خط است که باید در هر خط، بخش متناظر از تمام ارقام به همراه فاصله‌های بین آن‌ها چاپ شود.

مثال

ورودی نمونه ۱

123

خروجی نمونه ۱

```
      *****      *****
*           *           *
*           *           *
*           *           *
      *****      *****
*  *           *
*  *           *
*  *           *
      *****      *****
```

در این مثال:

- رقم ۱: فقط سگمنت‌های سمت راست
- رقم ۲: شکل موج‌دار
- رقم ۳: شبیه ۲ اما با سمت راست پایین

بین هر دو رقم ۳ فاصله وجود دارد.

برای درک بهتر می‌توانید از [این سایت](#) استفاده کنید.

ورودی نمونه ۲

9876543210

خروجی نمونه ۲

```
*****      *****      *****      *****      *****      *****      *****
*      *      *      *      *      *      *      *      *      *      *
*      *      *      *      *      *      *      *      *      *      *
*      *      *      *      *      *      *      *      *      *      *
*****      *****      *****      *****      *****      *****      *****
      *      *      *      *      *      *      *      *      *      *
      *      *      *      *      *      *      *      *      *      *
      *      *      *      *      *      *      *      *      *      *
*****      *****      *****      *****      *****      *****      *****
```

ورودی نمونه ۳

0

خروجی نمونه ۳

```
*****
*      *
*      *
*      *

*      *
*      *
*      *

*****
```

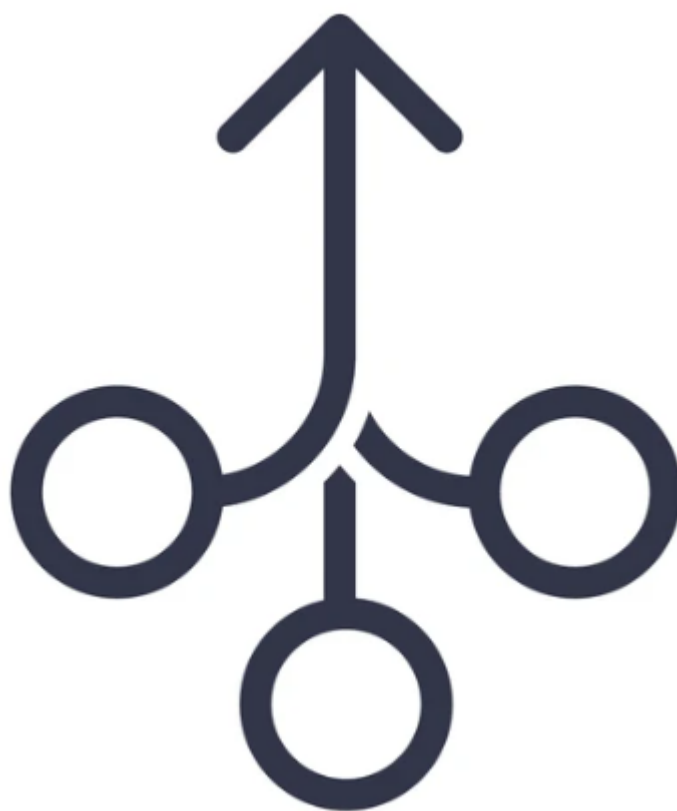
محدودیت‌ها و نکات

- ابعاد ثابت: هر رقم باید دقیقاً در ۹ سطر و ۷ ستون نمایش داده شود.
- کاراکترها: فقط از '*' برای بخش‌های روشن و ' ' (فاصله) برای بخش‌های خاموش استفاده شود.

میکس

- محدودیت زمان: ۳۰۰ میلی‌ثانیه
- محدودیت حافظه: ۱۰ مگابایت

در این سؤال، نباید در کد خود از براکت یعنی کاراکترهای [یا] استفاده کنید. در صورت وجود براکت در کد شما، نمره‌ی این سؤال به شما تعلق نمی‌گیرد.



شما باید تابعی بنویسید که دقیقاً به شکل زیر باشد:

```
1 | #include <stdlib.h>
2 |
3 | int* aggregator(int* a, int* b) {
4 |     // Your code goes here...
5 | }
```

ورودی a این تابع، پوینتری است که به اولین عضو یک آرایه از اعداد اشاره می‌کند. شما طول این آرایه را نمی‌دانید، اما می‌دانید که a_n عضو آخر این آرایه است، اگر و تنها اگر رابطه‌ی زیر برقرار باشد:

$$\sum_{i=0}^{n-1} a_i = a_n$$

طول آرایه حداقل ۱ و حداکثر ۱۰۰۰ است. اعضای آرایه، در حافظه پشت سر هم قرار گرفته‌اند.

ورودی b نیز به اولین عضو یک آرایه‌ی دیگر از اعداد اشاره می‌کند و تمام ویژگی‌هایی که برای آرایه‌ی قبلی گفته شد را دارد. خروجی تابع، باید آرایه‌ای باشد که حاصل از قرارگیری یکی‌درمیان اعضای دو آرایه‌ی ورودی است. به نمونه‌ها توجه کنید.

مثال

در نمونه‌ها منظور این است که تابع aggregator را صدا زده‌ایم، به‌عنوان ورودی به آن پوینترهایی داده‌ایم که به اولین عضو دو آرایه‌ی ذکرشده اشاره می‌کنند و انتظار داریم تابع، یک پوینتر برگرداند که به اولین عضو آرایه‌ی جواب اشاره کند.

ورودی نمونه ۱

```
2 -18 7 -9
4 6 8 18
```

خروجی نمونه ۱

```
2 4 -18 6 7 8 -9 18
```

ورودی نمونه ۲

```
7 0 4 0 7 8 4 5 6 2 10 53
6 7 1 4 0 10 6 2 7 43
```

خروجی نمونه ۲

7 6 0 7 4 1 0 4 7 0 8 10 4 6 5 2 6 7 2 43 10 53

اگر مثل این نمونه طول دو آرایه برابر نبود، هنگامی که اعضای یک آرایه تمام شد ادامه‌ی آرایه‌ی دیگر را پشت‌سر هم در آرایه‌ی پاسخ قرار دهید.

نکات مهم

- نام تابع و ورودی‌های آن را مطابق سؤال بنویسید.
- در کدی که ارسال می‌کنید نباید تابع `main` وجود داشته باشد و نباید هیچ چیزی پرینت شود. داوری کوئرا، تابع aggregator شما را مستقیماً صدا می‌زند و خروجی آن را بررسی می‌کند.
- قبل از ارسال، فرمت کد خود را از `c` به `cpp` تغییر دهید. (شما باید به زبان C کد بنویسید و صرفاً پیش از ارسال پسوند فایل را عوض کنید. این کار برای داوری سؤال مورد نیاز است.)
- به بازه‌ی `int` دقت کنید. به بازه‌ی طول آرایه دقت کنید. طول دو آرایه ممکن است متفاوت باشد.
- در کدتان نباید [یا] وجود داشته باشد.

مقدار درخت

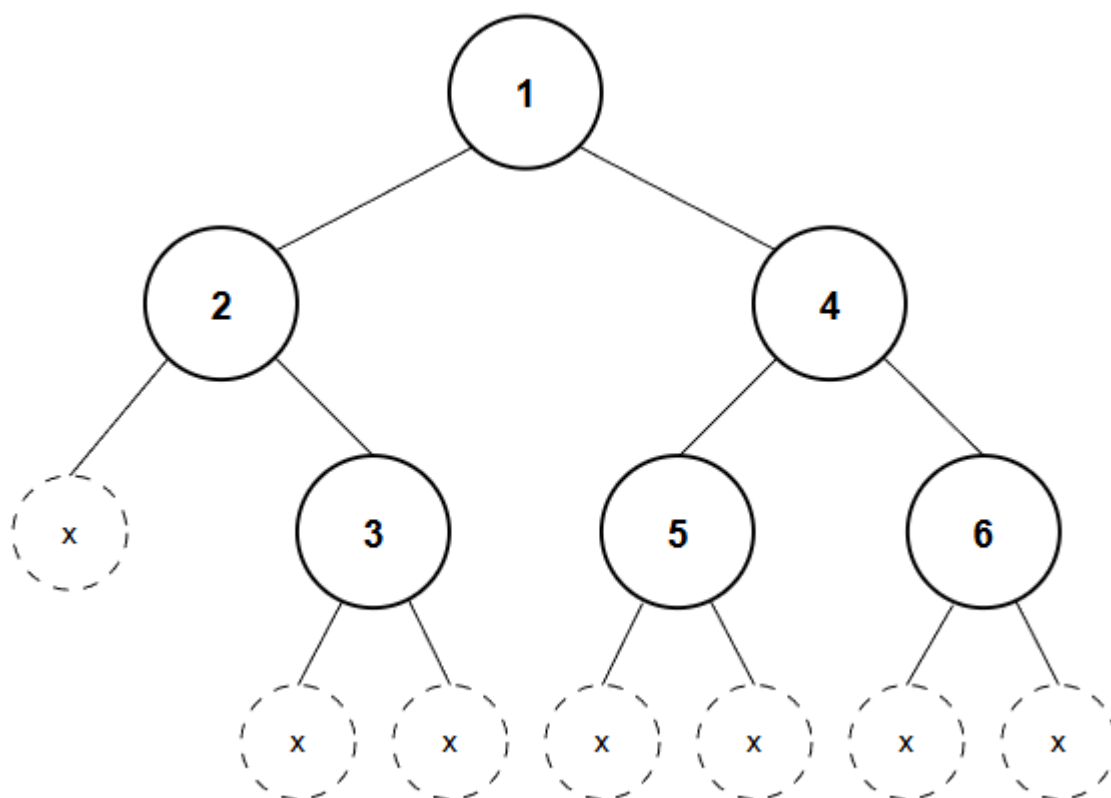
- محدودیت زمان: ۳۰۰ میلی‌ثانیه
- محدودیت حافظه: ۱۰ مگابایت

می‌خواهیم با ورودی گرفتن یک **درخت دودویی یا باینری**، مقداری را از آن استخراج کنیم و خروجی دهیم. درخت باینری، درختی است که هر گره‌اش می‌تواند حداکثر دو فرزند داشته باشد.

یکی از راه‌های پیمایش درخت باینری، **پیمایش پیش‌ترتیب (Preorder)** است، به این معنا که با شروع از ریشه‌ی درخت، ابتدا مقدار همان گره نوشته می‌شود، سپس فرزند چپ و بعد هم فرزند راست آن پیمایش می‌شود؛ و تا جایی ادامه پیدا می‌کند که به برگ (گره بدون فرزند) برسیم.

در این سؤال، تمام برگ‌ها با x مشخص شده‌اند تا بتوانید با ورودی گرفتن پیمایش پیش‌ترتیب، درخت را بسازید. برای این کار، پیشنهاد می‌شود از `struct` در زبان C استفاده کنید.

برای درک بهتر، به [این سایت](#) یا مثال زیر توجه کنید:



در این درخت، حاصل پیمایش پیش‌ترتیب به این صورت است:

1 2 x 3 x x 4 5 x x 6 x x

دقت کنید که مقادیر x صرفاً برای نمایش برگ‌های درخت آمده‌اند و در ادامه، در محاسباتان تأثیری ندارند.

مقدار موردنظر، این‌گونه از درخت به‌دست می‌آید:

- گره‌هایی که دو فرزند برگ دارند، مقدارشان برابر می‌شود با حاصل ضرب مقدار گره در مجموع مقدار فرزندهای آن گره؛ سپس فرزندان حذف می‌شوند.
- گره‌هایی که یک فرزند برگ دارند، مقدارشان برابر می‌شود با مجموع مقدار گره و دو برابر مقدار فرزند گره؛ سپس فرزند حذف می‌شود.
- این فرایند از برگ‌ها شروع می‌شود و آن‌قدر ادامه می‌یابد تا فقط ریشه‌ی درخت باقی بماند و مقدار ریشه، همان مقدار موردنظر است.

ورودی

ورودی تنها در یک خط می‌آید که همان حاصل پیمایش پیش‌ترتیب درخت باینری است. مقدار هر گره، عددی صحیح است و اگر آن را با n نشان دهیم، داریم:

$$-500 \leq n \leq 500$$

خروجی

شما باید پس از ساختن درخت موردنظر از روی حاصل پیمایش پیش‌ترتیب، مقدار استخراج‌شده از درخت را بیابید و در یک خط خروجی دهید.

راهنمایی

برای سهولت در ورودی گرفتن، پیشنهاد می‌شود از قطعه کد زیر برای شروع استفاده کنید. محدودیت‌های تعداد کاراکتر در این قطعه کد رعایت شده‌است.

متغیر `preorder` در کد زیر، نشان‌دهنده‌ی آرایه‌ای از مقادیر موجود در ورودی است که با فاصله از یکدیگر جدا شده‌اند.

همچنین در صورت نیاز برای تبدیل متغیرهای رشته‌ای که حاوی عدد هستند به متغیرهای عددی، می‌توانید از تابع `atoi` استفاده کنید.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  // You can define desired functions...
6
7  int main() {
8      char line[5000];
9      if (!fgets(line, sizeof(line), stdin)) {
10         return 1;
11     }
12
13     char *preorder[1000];
14     int count = 0;
15     char *token = strtok(line, " \n");
16     while (token != NULL) {
17         preorder[count++] = token;
18         token = strtok(NULL, " \n");
19     }
20
21     // Your code goes here...
22
23     return 0;
24 }
```

مثال

ورودی نمونه ۱

1 2 x 3 x x 4 5 x x 6 x x

خروجی نمونه ۱

52

ورودی نمونه ۲

-10 2 -3 x x x x

خروجی نمونه ۲

-18

